

JPA Architecture & Entity Lifecycle

What is an Entity?

An **Entity** is a Java class that represents a **table in the database**.

Each object of that class represents **one row**.

But in JPA, an entity is not just a simple object.

Once it is managed, it becomes a **tracked object** whose changes are automatically synchronized with the database.

Where Spring Data JPA fits

Spring Data JPA is **not JPA itself**.

It is only a **layer on top of JPA** that gives us ready-made methods like `save()`, `findById()`, `delete()`.

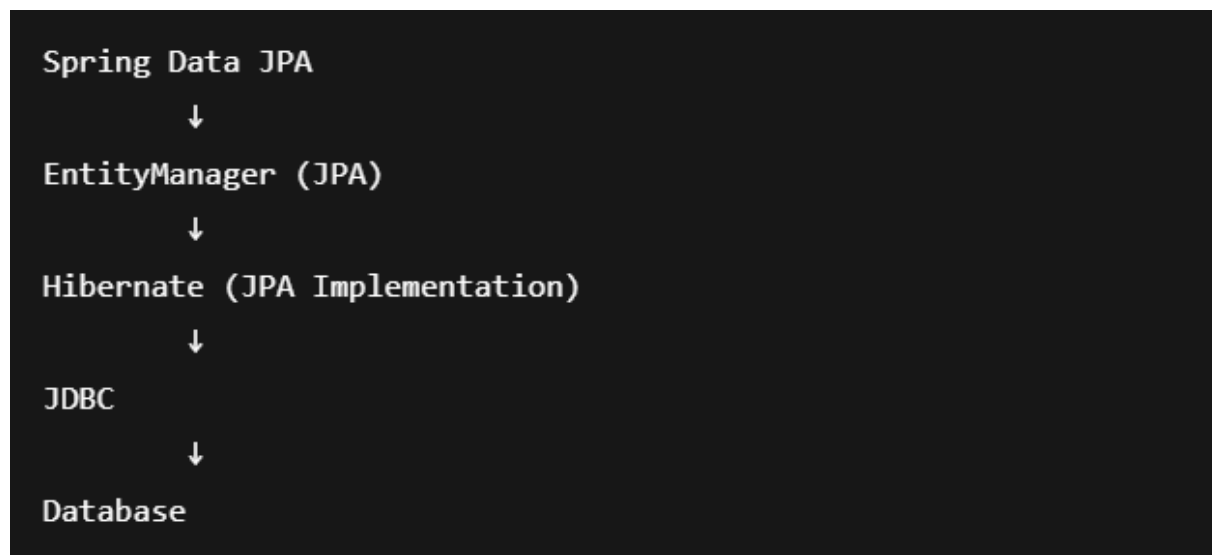
Internally, Spring Data JPA always talks to **JPA's EntityManager**.

JPA itself does not talk to the database directly.

It uses **Hibernate** as its internal implementation.

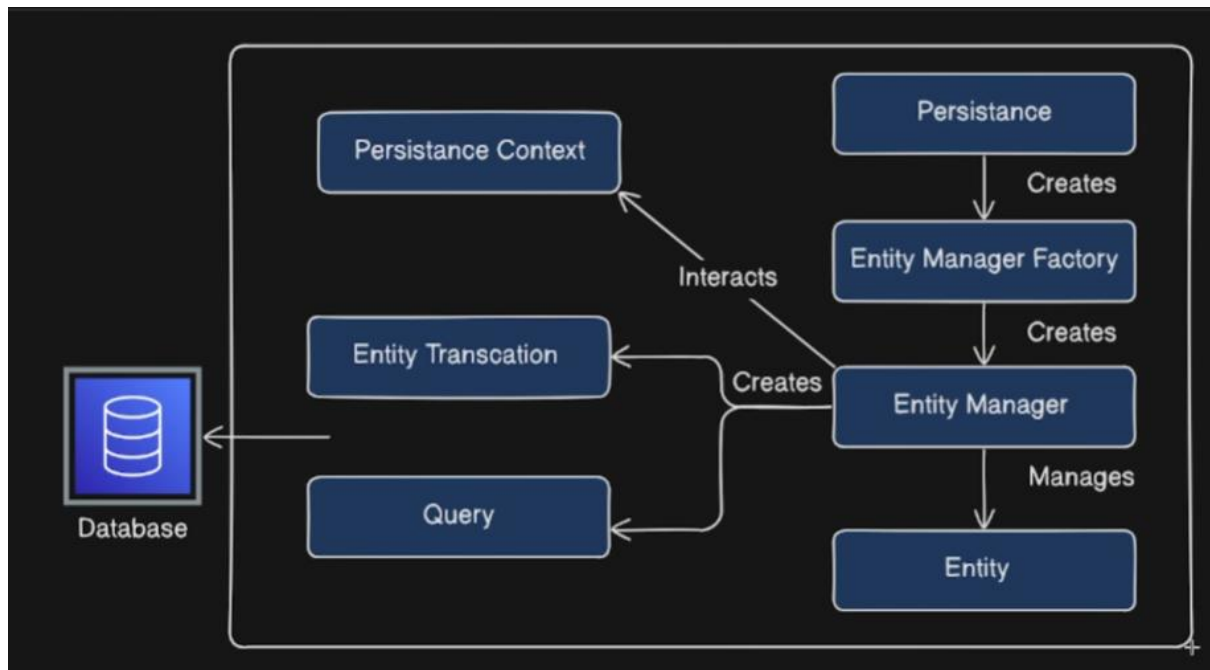
Hibernate finally uses **JDBC** to execute SQL.

So the real flow is:



JPA Architecture – The Journey of an Entity

When your application starts, **Persistence** (configuration) creates an **EntityManagerFactory**. This factory is responsible for creating **EntityManager** objects.



1. Entity Manager – The Controller of Entities

The **EntityManager** is an interface provided by JPA.

It is responsible for **managing the entire life of an entity** inside the application.

All operations you perform on an entity are handled by the EntityManager, such as:

- `persist()` – save
- `find()` – fetch
- `merge()` – update
- `remove()` – delete
- `flush()` – sync to DB

However, the EntityManager **does not talk to the database directly**.

It only **coordinates and controls** what should happen.

2. Entity Manager Factory – Creator of Entity Managers

The **EntityManagerFactory** is responsible for creating **EntityManager** instances.

- It is created **once at application startup**
- It is **heavy and expensive**
- From this factory, multiple **EntityManagers** can be created

The factory itself is created by **Persistence** (configuration).

3. Persistence – The Configuration Layer

Persistence is responsible for:

- Reading DB configuration
- Loading JPA properties
- Creating the **EntityManagerFactory**

In Spring Boot, this is handled using `application.yml` / `application.properties`.

So the startup flow is:

Persistence → **EntityManagerFactory** → **EntityManager**

4. Persistence Context – The Memory of JPA

The **Persistence Context** is an **in-memory cache**.

It is a temporary storage area where **managed entities are kept**.

Think of it as a **bag of tracked entities**.

The **EntityManager** never works directly on the database first — it always interacts with the **Persistence Context**.

EntityManager ↔ **Persistence Context**

Any entity inside the **Persistence Context** is called a **Managed Entity**.

5. Why Entity Manager Cannot Talk to DB Directly

The **EntityManager** is only an **interface (abstraction layer)**. It cannot generate SQL or connect to the database by itself.

So it creates:

- **Query objects**
- **EntityTransaction objects**

And offloads the real work to its implementation.

6. Hibernate – The JPA Implementation

We already know that **Hibernate is the internal implementation of JPA.**

So when EntityManager receives a command:

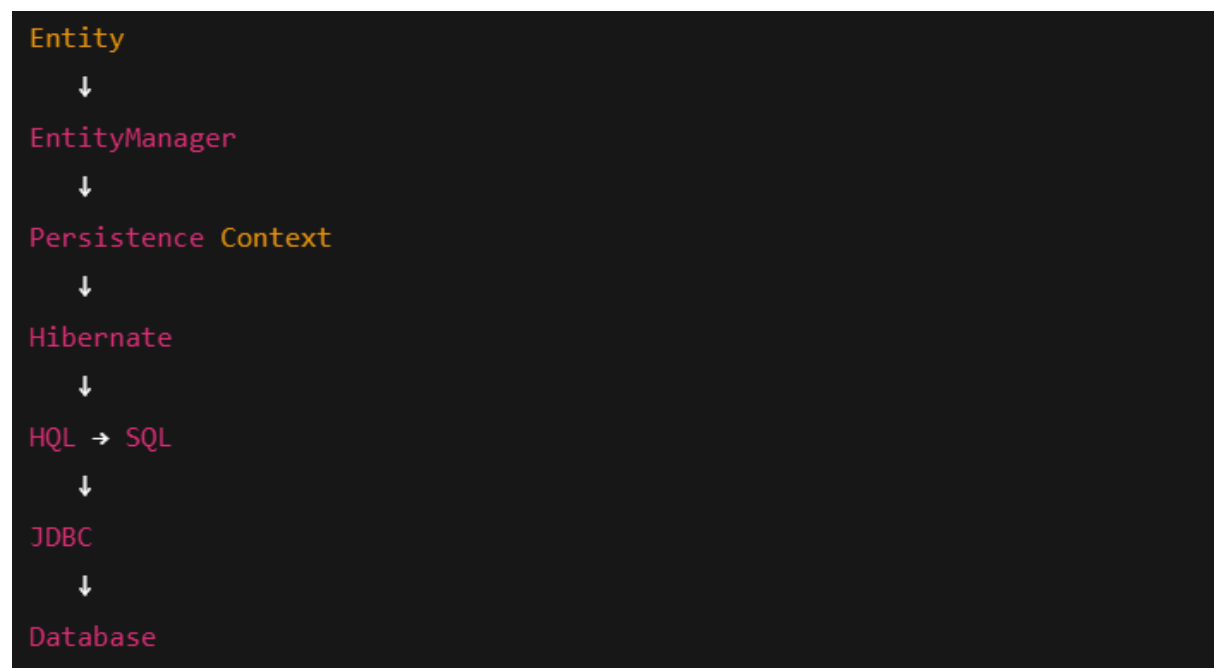
```
entityManager.persist(entity);
```

It forwards the request to **Hibernate**.

Hibernate then:

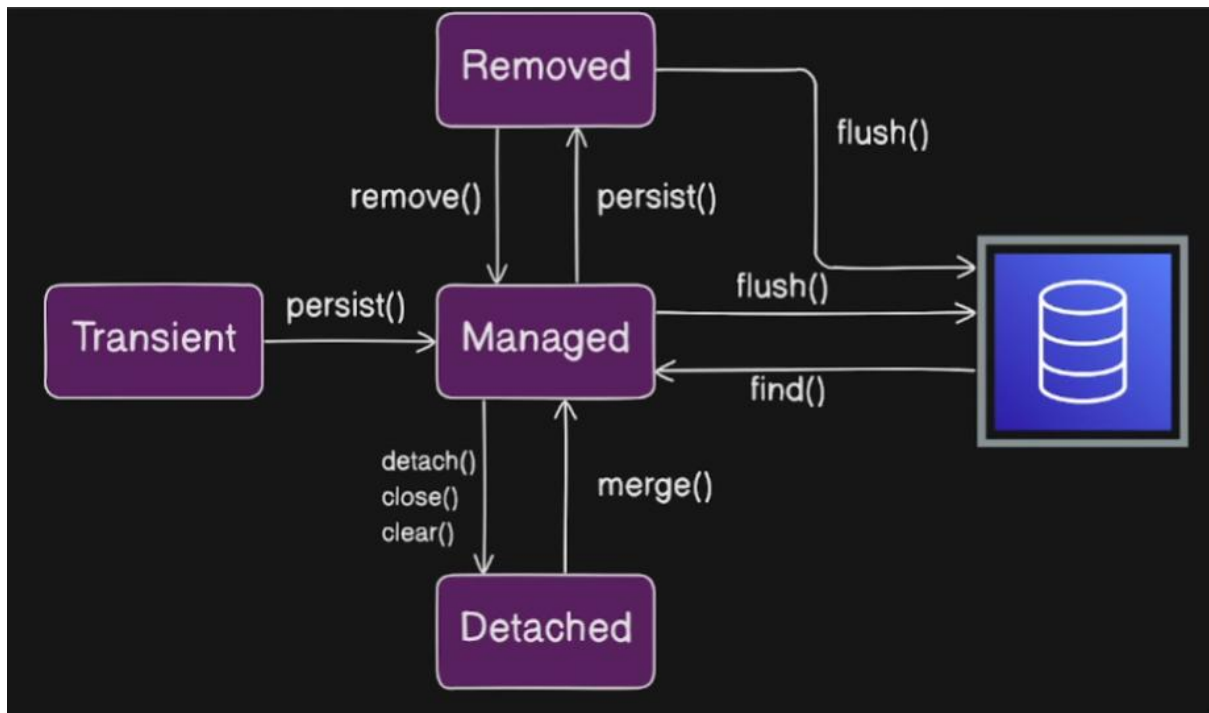
1. Creates **HQL**
2. Converts HQL → **SQL**
3. Sends SQL to the database using **JDBC**

7. Complete Journey of an Entity



This is the real path every entity follows in a Spring Boot application.

Entity Lifecycle – The Movement of an Entity



In JPA, every entity object passes through a **fixed set of states** during its life inside the application.

These states describe **how JPA and the EntityManager are controlling that object**.

Every entity passes through **four states**:

`Transient` → `Managed` → `Detached` → `Removed`

1. Transient State – New Object

The moment we create an entity object in our application:

```
Employee emp = new Employee();
```

The entity is in the **Transient state**.

At this stage:

- It is a fresh Java object
- It is NOT managed by the EntityManager
- It is NOT inside the Persistence Context
- It does NOT exist in the database

It is just a normal object in memory.

2. Persist → Managed State

When we call:

```
entityManager.persist(emp);
```

The entity moves from **Transient** → **Managed**.

Now:

- The entity is controlled by the EntityManager
- It is added into the **Persistence Context**
- It becomes a **tracked object**

Once an entity is in the **Managed state**, any change you make to it is automatically monitored.

3. Managed State – Persistence Context Control

While the entity is managed:

- It is stored inside the Persistence Context
- Entity Manager interacts with the Persistence Context to manage it
- Persistence Context stores all managed entities

This is why JPA can detect changes automatically.

4. Flush → Database Synchronization

When we call:

```
entityManager.flush();
```

or when the transaction commits:

- Hibernate generates SQL
- SQL is executed on the database
- Data is saved in the database

Flush means **synchronize memory with the database**.

5. Find

When we call:

```
entityManager.find(Employee.class, 1L);
```

- JPA first checks the Persistence Context
- If not found, it fetches from the database
- The result is added to the Persistence Context
- The entity becomes **Managed**

6. Detached State

If we call:

```
entityManager.detach(emp);  
entityManager.clear();  
entityManager.close();
```

The entity moves from **Managed** → **Detached**.

Now:

- The entity is no longer tracked
- It was once managed
- Any changes will NOT be synchronized to the database

7. Merge → Back to Managed

To make a detached entity managed again:

```
entityManager.merge(emp);
```

The entity moves from **Detached** → **Managed**.

8. Removed State

When we call:

```
entityManager.remove(emp);
```

The entity moves from **Managed** → **Removed**.

It is marked for deletion.

9. Persist Again

```
entityManager.persist(emp);
```

The entity moves from **Removed** → **Managed** again.

10. Final Delete – Flush

```
entityManager.flush();
```

The row is finally deleted from the database.

Lifecycle Flow

<code>new Entity()</code>	→ Transient
<code>persist()</code>	→ Managed
<code>flush/commit</code>	→ Database Sync
<code>detach/clear/close</code>	→ Detached
<code>merge()</code>	→ Managed
<code>remove()</code>	→ Removed
<code>persist()</code>	→ Managed
<code>flush()</code>	→ Database Delete